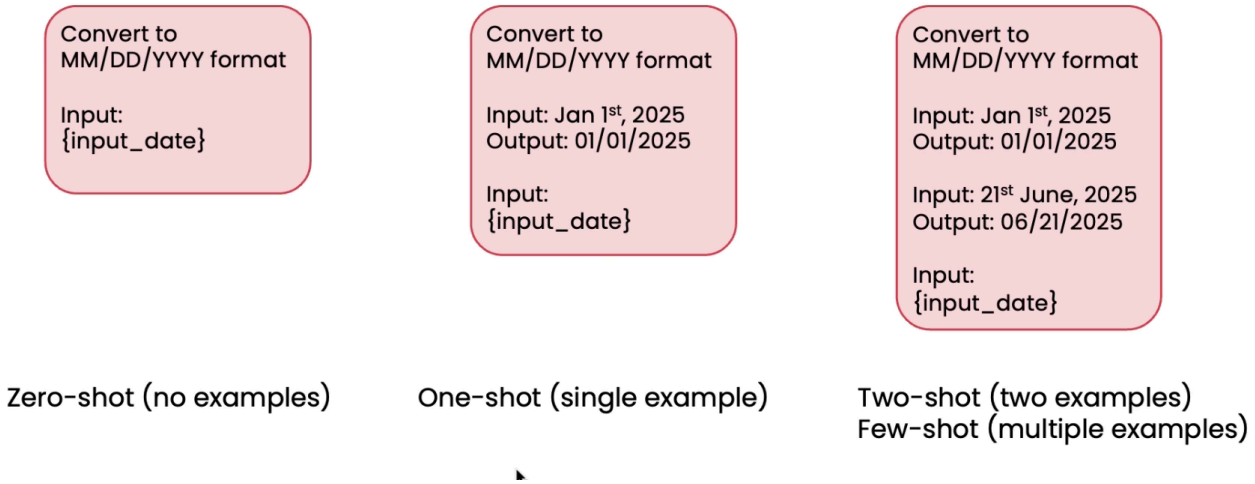
Reflection Design pattern

Prompting

Direct Generation



Reflection Prompting

It is a technique where you ask an AI model to review and critique its own output (or sometimes we can use a different model (reasoning model is strongly recommended!) to judge the 1st model) before giving the final answer. It's essentially building a self-correction loop into the prompt.

But, Reflection consistently outperforms direct generation on a variety of tasks.

How it works
The basic idea is to have the model go through two (or more) passes

1. **First pass**: Generate an initial response
2. **Reflection pass**: Review that response, identify errors, gaps, or improvements
3. **Final pass**: Produce a revised, better answer based on that self-critique
   Example: Write a Python function to reverse a linked list. Then review your solution for correctness, edge cases, and efficiency. Finally, rewrite it if needed based on your review.

Tips for writing the reflection prompt

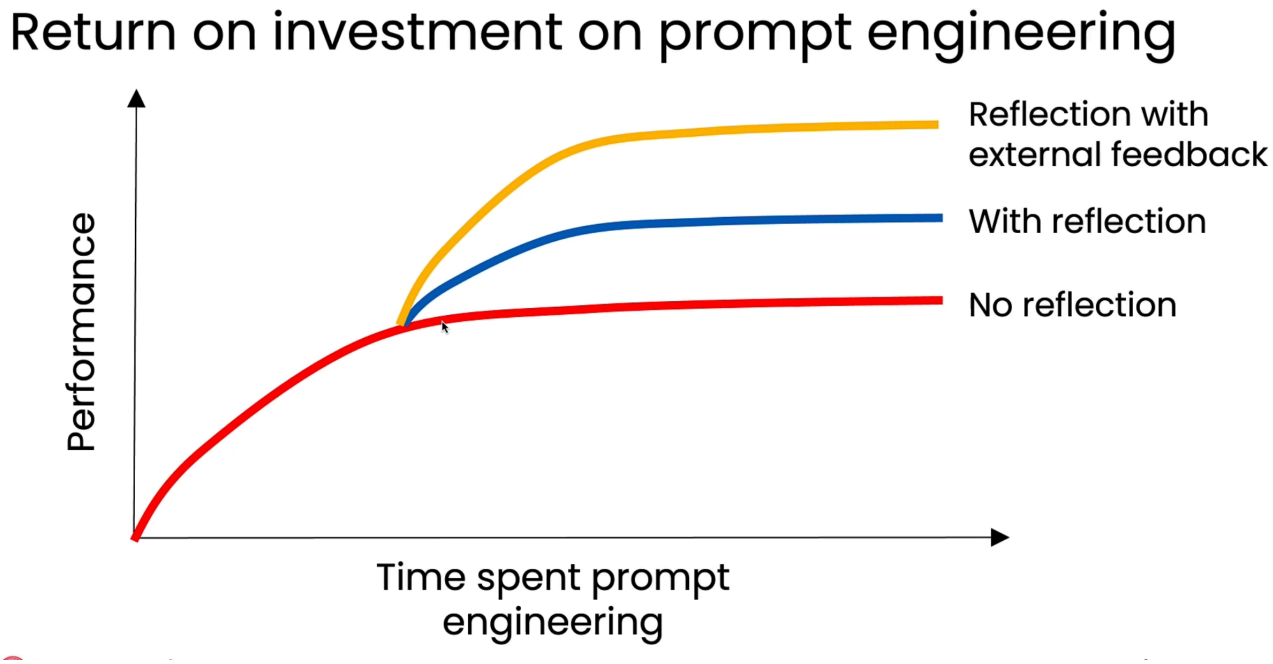
- Clearly indicate the reflection action
- Specify criteria to check

Evaluate the impact of reflection

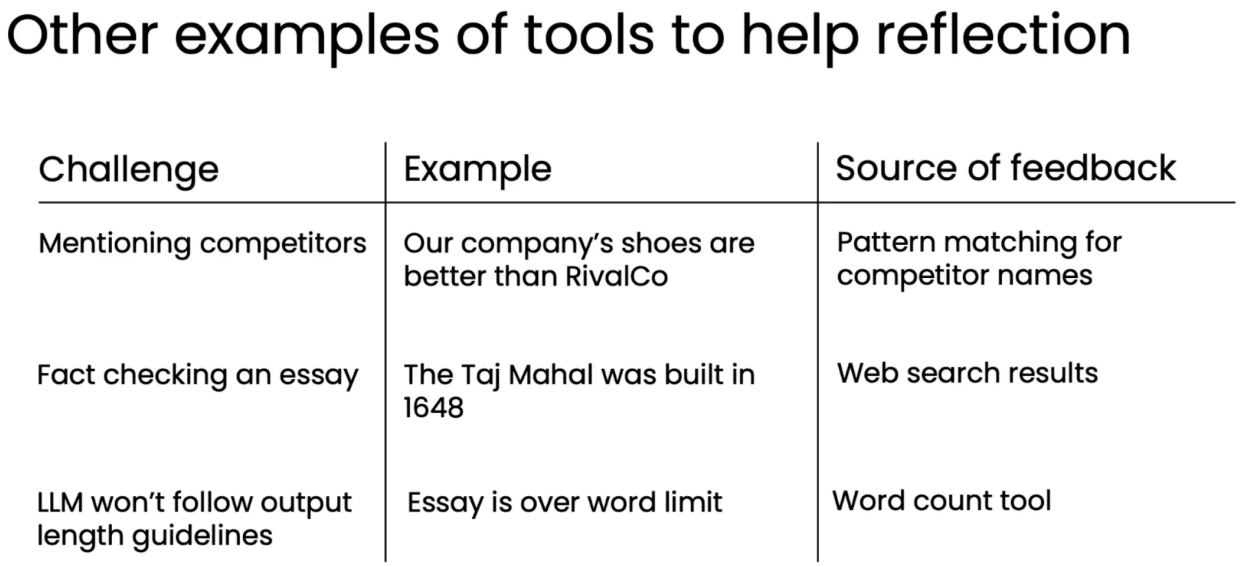
- **Objective evals:**
  - code-based evals are easier

    - build a dataset of ground truth examples
- Subjective evals:
    - use LLM as a judge
    - Rubric-based grading is better

External Feedback Reflection



External feedback reflection prompting is a variation where, instead of the model critiquing **its own output**, it receives **feedback from an outside source** and then reflects and revises based on that.




```
def run_sql_workflow(
    db_path: str,
    question: str,
```

```
    model_generation: str = "openai:gpt-4.1",
    model_evaluation: str = "openai:gpt-4.1",
):
    """
    End-to-end workflow to generate, execute, evaluate, and refine SQL queries.

    Steps:
      1) Extract database schema
      2) Generate SQL (V1)
      3) Execute V1 → show output
      4) Reflect on V1 with execution feedback → propose refined SQL (V2)
      5) Execute V2 → show final answer
    """

    # 1) Schema
    schema = utils.get_schema(db_path)
    utils.print_html(
        schema,
        title="📘 Step 1 — Extract Database Schema"
    )

    # 2) Generate SQL (V1)
    sql_v1 = generate_sql(question, schema, model_generation)
    utils.print_html(
        sql_v1,
        title="🧠 Step 2 — Generate SQL (V1)"
    )

    # 3) Execute V1
    df_v1 = utils.execute_sql(sql_v1, db_path)
    utils.print_html(
        df_v1,
        title="🧪 Step 3 — Execute V1 (SQL Output)"
    )

    # 4) Reflect on V1 with execution feedback → refine to V2
    feedback, sql_v2 = refine_sql_external_feedback(
        question=question,
        sql_query=sql_v1,
        df_feedback=df_v1,          # external feedback: real output of V1
        schema=schema,
        model=model_evaluation,
    )
    utils.print_html(
        feedback,
        title="🧭 Step 4 — Reflect on V1 (Feedback)"
    )
    utils.print_html(
        sql_v2,
        title="🔁 Step 4 — Refined SQL (V2)"
```

```
)

# 5) Execute V2
df_v2 = utils.execute_sql(sql_v2, db_path)
utils.print_html(
    df_v2,
    title="✅ Step 5 — Execute V2 (Final Answer)"
)
```